

Correction

TD — Administration Linux

Chapitre 1 — Présentation et objectifs

Chapitre 2 — Généralités sur les systèmes Unix/Linux

Chapitre 3 — Installations

Chapitre 4 — Shell, commandes et aide en ligne

Chapitre 5 — Manipulation du système de fichiers

Chapitre 6 — Édition et traitement de données textuelles

Chapitre 7 — Quelques outils de traitement de données

Chapitre 8 — Introduction à la programmation shell Bash

Année universitaire 2026–2027

1 Présentation et objectifs

Correction 1.1 — QCM : Historique et philosophie de Linux

1. Linux a été créé par : **b) Linus Torvalds** — Linus Torvalds a créé le noyau Linux en 1991. Richard Stallman est le fondateur du projet GNU (1983). Dennis Ritchie et Ken Thompson sont les créateurs d'Unix.
2. La licence GPL stipule que : **b) Le logiciel est libre** : utilisation, modification et redistribution sont autorisées. — La GPL garantit les quatre libertés du logiciel libre.
3. Le projet GNU a été lancé en : **a) 1983** — Richard Stallman a lancé le projet GNU en 1983. Linux (noyau) est apparu en 1991.
4. L'objectif principal du système GNU/Linux est : **b) Fournir un système d'exploitation libre et ouvert** — GNU/Linux vise à offrir une alternative libre aux systèmes propriétaires.

Correction 1.2 — Vrai ou Faux : Distributions et licences

1. **Faux.** Linux est un noyau uniquement. Le système complet est GNU/Linux (noyau + outils GNU + applications). Dire « Linux » pour le système complet est un abus de langage courant.
2. **Vrai.** Ubuntu, Debian et Fedora sont trois distributions Linux parmi les plus populaires.
3. **Faux.** Certaines distributions ont des versions payantes (ex. : Red Hat Enterprise Linux, SUSE Linux Enterprise), même si le code source reste libre.
4. **Vrai.** Le noyau Linux est publié sous licence GPL v2 (et uniquement v2), comme l'a choisi Linus Torvalds.
5. **Faux.** Libre $\neq$ gratuit. « Libre » signifie accès au code source et liberté de modification/redistribution. Un logiciel libre peut être vendu (ex. : RHEL).

Rappel — Libre vs Gratuit

« Libre » (free software) fait référence aux libertés de l'utilisateur (liberté 0 à 3 de la FSF), pas au prix. En anglais, « free as in freedom, not as in free beer ».

Correction 1.3 — Tableau comparatif : Licences et philosophie

Licence	Source ouverte	Modification autorisée	Redistribution libre
GPL v2	Oui	Oui	Oui (mêmes conditions)
MIT	Oui	Oui	Oui (avec clause de copyright)
Apache 2.0	Oui	Oui	Oui (avec clause de brevets)
Propriétaire (Windows)	Non	Non	Non
BSD	Oui	Oui	Oui (sans contrainte de licence)

Remarque — Différence majeure GPL vs BSD/MIT

La GPL est une licence « copyleft » : toute œuvre dérivée doit conserver la même licence. Les licences BSD et MIT sont « permissives » : elles autorisent l'intégration dans des projets propriétaires.

2 Généralités sur les systèmes Unix/Linux**Correction 2.1 — QCM : Architecture Unix/Linux**

1. Le noyau Linux gère : **b) La mémoire, les processus et les pilotes matériels** — Le noyau est le cœur du système, il gère les ressources matérielles.
2. L'architecture Unix est : **b) En couches** — L'architecture classique : matériel → noyau → shell → applications.
3. Le shell est : **b) Un interpréteur de commandes** — Il traduit les commandes de l'utilisateur en appels système vers le noyau.
4. Le mode mono-utilisateur signifie que : **b) Le système démarre en mode maintenance sans réseau** — C'est un mode de dépannage (runlevel 1 / rescue mode).

Correction 2.2 — Vrai ou Faux : Concepts fondamentaux

1. **Vrai.** Unix a été créé en 1969–1970 par Ken Thompson et Dennis Ritchie aux laboratoires Bell (AT&T).
2. **Vrai.** Linux est multi-utilisateur (plusieurs comptes) et multitâche (plusieurs processus simultanés).
3. **Faux.** Le noyau Linux est monolithique (avec modules chargeables dynamiquement), pas un micro-noyau.
4. **Vrai.** Sous Linux, tout est fichier : les périphériques (/dev), les processus (/proc), les sockets, etc.
5. **Vrai.** Le répertoire racine est /, contrairement à Windows qui utilise des lettres de lecteur (C:\, D:\, etc.).
6. **Vrai.** Un processus est un programme en cours d'exécution, avec son espace mémoire et ses ressources.

Correction 2.3 — Architecture en couches

Élément	Couche
Noyau Linux	Noyau
Bash	Shell
Firefox	Applications utilisateur
Pilote réseau (module)	Noyau
X11 / Wayland	Système de fenêtrage
GCC	Applications utilisateur
systemd	Noyau (service système) / Applications

Remarque — Pilotes et modules

Les pilotes matériels sous Linux sont souvent des modules du noyau (fichiers .ko) chargés dynamiquement. Ils font partie de la couche noyau même s'ils sont chargeables à la demande.

Correction 2.4 — Arborescence FHS

Répertoire	Rôle
/bin	Commandes exécutables essentielles (ls, cp, mv, rm, etc.)
/etc	Fichiers de configuration du système
/home	Répertoires personnels des utilisateurs
/var	Données variables (logs, spool d'impression, bases de données)
/usr	Programmes et bibliothèques utilisateur
/tmp	Fichiers temporaires (effacés au redémarrage)
/dev	Fichiers spéciaux représentant les périphériques
/proc	Système de fichiers virtuel (informations sur les processus et le noyau)
/opt	Logiciels tiers/additionnels
/root	Répertoire personnel du super-utilisateur (root)

Correction 2.5 — Types de fichiers sous Linux

1. - : Fichier ordinaire (régulier)
2. d : Répertoire (directory)
3. l : Lien symbolique (soft link)
4. b : Fichier spécial en mode bloc (ex. : disque dur /dev/sda)
5. c : Fichier spécial en mode caractère (ex. : terminal /dev/tty1)
6. s : Socket (communication réseau entre processus)
7. p : Tube nommé (named pipe / FIFO)

Remarque — Identification

On peut voir le type de fichier avec `ls -la`. Le premier caractère de chaque ligne indique le type : -, d, l, b, c, s ou p.

3 Installations

Correction 3.1 — QCM : Méthodes d'installation

1. Pour installer Ubuntu : **d) Toutes ces réponses** — USB bootable, DVD et PXE sont tous des méthodes d'installation valides.
2. Partitionnement recommandé : **b) /boot, /, swap** — Un serveur nécessite au minimum une partition boot, une racine et un swap.
3. Le swap sert à : **b) Étendre la mémoire vive sur le disque dur** — Le swap est une zone d'échange sur disque utilisée quand la RAM est pleine.
4. Un gestionnaire de paquets est : **b) Un logiciel qui installe, met à jour et supprime des paquets**

logiciels. — Il gère les dépendances automatiquement (ex. : apt, dnf, pacman).

Correction 3.2 — Vrai ou Faux : Partitionnement et amorçage

1. **Vrai.** GRUB2 est le chargeur d'amorçage le plus courant sous Linux. Il permet de choisir le noyau ou l'OS au démarrage.
2. **Faux.** /boot contient le noyau et les images initramfs, pas les fichiers personnels (qui sont dans /home).
3. **Vrai.** ext4 est le système de fichiers par défaut de la plupart des distributions Linux.
4. **Vrai.** UEFI a remplacé le BIOS traditionnel et supporte les tables de partitions GPT (plus de 4 partitions primaires, disques > 2 To).
5. **Faux.** Linux peut être installé en dual-boot avec Windows sur le même disque (ou des disques séparés), via GRUB2.
6. **Vrai.** lsblk liste les périphériques bloc avec leurs points de montage.

Correction 3.3 — Plan de partitionnement

	Point de montage	Système de fichiers	Taille suggérée
	/boot	ext4	1 Go
	/	ext4	30 Go
Proposition pour un serveur avec 100 Go :	/home	ext4	40 Go
	/var	ext4	20 Go
	swap	swap	4 Go

Remarque — Taille du swap

La règle classique : $\text{swap} \approx \text{RAM}$ pour les serveurs avec ≤ 8 Go de RAM. Au-delà, le swap peut être plus petit (2–4 Go) ou égal à la RAM selon l'usage (hibernation, bases de données).

Correction 3.4 — Gestionnaires de paquets

Distribution	Gestionnaire	Commande d'installation
Debian / Ubuntu	APT / dpkg	sudo apt install paquet
Fedora / RHEL	DNF / RPM	sudo dnf install paquet
Arch Linux	Pacman	sudo pacman -S paquet
openSUSE	Zypper / RPM	sudo zypper install paquet
Alpine	APK	apk add paquet

Remarque — Familles de distributions

Debian et Red Hat sont les deux grandes familles. Les distributions dérivées utilisent le même gestionnaire de paquets : Ubuntu (Debian), CentOS/Rocky (RHEL), Manjaro (Arch).

4 Shell, commandes et aide en ligne

Correction 4.1 — QCM : Le shell

1. Shell par défaut : **b)** `bash` — Bash (Bourne Again SHell) est le shell par défaut de la plupart des distributions. Note : Ubuntu utilise désormais `dash` pour `/bin/sh` et `zsh` peut être choisi.
2. Version de Bash : **b) et d)** — `bash --version` affiche la version. `echo $BASH_VERSION` fonctionne aussi. `which bash` donne le chemin, pas la version.
3. Le shell est : **b) Un interpréteur de commandes** — Il lit, analyse et exécute les commandes de l'utilisateur.
4. Changer de shell : **a)** `chsh` — `chsh` (change shell) modifie le shell de connexion dans `/etc/passwd`. Temporairement, on peut lancer un autre shell directement (ex. : `zsh`).

Correction 4.2 — Vrai ou Faux : Aide en ligne

1. **Vrai.** `man` affiche les pages du manuel système (formaté avec `less` par défaut).
2. **Vrai.** `man 5 passwd` affiche la section 5 (Formats de fichiers) pour `/etc/passwd`, tandis que `man 1 passwd` affiche la commande.
3. **Faux.** `--help` affiche un résumé court des options, alors que `man` fournit une documentation complète et structurée.
4. **Vrai.** `apropos` (équivalent de `man -k`) recherche un mot-clé dans les titres et descriptions.
5. **Vrai.** `info` propose une navigation hypertexte avec des nœuds et des liens, plus riche que `man`.
6. **Vrai.** Les pages de manuel sont organisées en 9 sections (1 à 9, plus des sous-sections).

Correction 4.3 — Sections du manuel

Section	Contenu
1	Commandes utilisateur
2	Appels système
3	Bibliothèques
4	Fichiers de configuration (périphériques)
5	Formats de fichiers
6	Jeux
7	Miscellaneous (divers, conventions, protocoles)
8	Administrateur (commandes système)

Remarque — Sections les plus utilisées

Les sections 1, 5 et 8 sont les plus consultées. La section 1 pour les commandes, la 5 pour les fichiers de configuration, la 8 pour les commandes d'administration.

Correction 4.4 — Navigation dans le manuel

1. Pour chercher permission dans man chmod : taper /permission puis Entrée dans le visualiseur. — / lance une recherche vers le bas, ? vers le haut.
2. La section OPTIONS décrit les options de la commande. On peut y accéder en cherchant /OPTIONS ou en défilant.
3. Pour quitter : appuyer sur q.
4. man 5 passwd — On précise le numéro de section avant le nom.

Correction 4.5 — Différences entre shells

Caractéristique	sh	bash	zsh	fish
Shell POSIX par défaut	Oui	Non	Non	Non
Auto-complétion avancée	Non	Oui	Oui	Oui
Compatible bash	—	Oui	Oui	Non
			(largement)	
Syntaxe différente de bash	Non	—	Non	Oui
Shell par défaut Ubuntu	Non	Oui	Non	Non
		(historique)		

Remarque — fish et la compatibilité

Le shell fish (Friendly Interactive SHell) a une syntaxe différente de Bash (pas de = pour les tests, pas de && historiquement). Les scripts Bash ne sont pas directement compatibles avec fish.

5 Manipulation du système de fichiers**Correction 5.1 — QCM : Commandes de base**

1. Lister avec fichiers cachés : **b)** ls -a — ls sans option n'affiche pas les fichiers cachés (commençant par .). ls -l affiche les détails mais pas les cachés. dir n'est pas standard.
2. Répertoire courant : **b)** pwd — pwd (print working directory) affiche le chemin absolu du répertoire courant.
3. Créer avec parents : **b)** mkdir -p — L'option -p crée les répertoires parents si nécessaires (pas d'erreur s'ils existent).
4. rm -rf : **b)** **Supprime récursivement et sans confirmation** — -r = récursif, -f = forcé (pas de confirmation). Très dangereux !
5. Copier un répertoire : **b)** cp -r dossier1 dossier2 — L'option -r (ou -R) est obligatoire pour copier des répertoires.

Remarque — Danger de rm -rf

La commande rm -rf supprime irrémédiablement les fichiers (pas de corbeille). Une erreur classique : rm -rf / ou rm -rf * dans le mauvais répertoire. Toujours vérifier le chemin avant d'exécuter.

Correction 5.2 — Vrai ou Faux : Permissions

1. **Vrai.** Chaque fichier a un propriétaire (owner), un groupe (group) et des permissions pour chacun (r, w, x).
2. **Faux.** La permission r sur un répertoire permet de lister son contenu (ls). C'est x qui permet d'y entrer (cd).
3. **Vrai.** La permission x (exécution) sur un fichier permet de l'exécuter comme programme. Pour un répertoire, elle permet d'y accéder.
4. **Vrai.** chmod 755 = rwxr-xr-x : propriétaire (7=rwx), groupe (5=r-x), autres (5=r-x).
5. **Faux.** chown modifie le propriétaire. Pour modifier le groupe, on utilise chgrp ou chown user: groupe.
6. **Vrai.** umask définit les permissions par défaut retirées lors de la création de fichiers (ex. : 022 donne 644 pour les fichiers, 755 pour les répertoires).
7. **Vrai.** Les 3 groupes sont propriétaire (u), groupe (g) et autres (o).

Correction 5.3 — Calcul de permissions

1. chmod 644 → rw-r--r-- — 6=rw-, 4=r--, 4=r--
2. chmod 755 → rwxr-xr-x — 7=rwx, 5=r-x, 5=r-x
3. chmod 700 → rwx----- — 7=rwx, 0=---, 0=---
4. chmod 777 → rwxrwxrwx — 7=rwx pour les trois groupes
5. chmod 640 → rw-r---- — 6=rw-, 4=r--, 0=---
6. chmod +x script.sh → rwxr-xr-x (si le fichier avait rw-r--r--) — Ajoute x pour tous.
7. chmod u+w,go-rwx secret.txt → rw----- — Ajoute w au propriétaire, retire rwx au groupe et aux autres.
8. chmod o=r fichier → Propriétaire et groupe inchangés, autres = r-- uniquement. — = fixe la permission exacte.

Remarque — Conversion octal symbolique

Chaque chiffre octal correspond à 3 bits : r=4, w=2, x=1. Somme = valeur. Ex. : r-x = 4+0+1 = 5.

Correction 5.4 — Commandes de manipulation de fichiers

Commande	Description
ls -la	Liste détaillée de tous les fichiers (y compris cachés)
cp	Copie de fichiers ou répertoires
mv	Déplace ou renomme des fichiers/répertoires
rm	Supprime des fichiers ou répertoires
touch	Crée un fichier vide ou met à jour la date de modification
mkdir	Crée un répertoire
rmdir	Supprime un répertoire vide
ln -s	Crée un lien symbolique
find	Recherche de fichiers selon critères
tree	Affiche l'arborescence sous forme d'arbre

Correction 5.5 — Recherche de fichiers

1. `find /home -name "rapport.pdf"`
2. `find /var/log -mtime -7`
3. `find /home -user alice`
4. `find / -size +100M`
5. `find /home -type d -name ".git"`
6. `find . -name "*.sh" -exec ls -la {} \;`

Remarque — Options de find

Principales options : `-name` (nom), `-type` (f=fichier, d=répertoire, l=lien), `-user` (propriétaire), `-size` (taille), `-mtime` (date de modification), `-perm` (permissions), `-exec` (exécuter une commande sur les résultats). Le `{ }` représente le fichier trouvé et `\;` termine la commande.

Correction 5.6 — Liens physiques et symboliques

1. **Lien physique** : le fichier reste accessible via le lien physique car il pointe vers le même inœud. Le contenu n'est supprimé que quand le dernier lien (physique) est effacé (compteur de liens = 0).
2. **Lien symbolique** : le lien devient « brisé » (dangling symlink) car il pointe vers un chemin qui n'existe plus. `ls -la` affichera le lien en rouge (selon la configuration).
3. Lien physique : `ln fichier_origine lien_physique` ; Lien symbolique : `ln -s cible lien_symbolique`
4. **Non**, on ne peut pas créer de lien physique sur un répertoire sous Linux (limitation historique pour éviter des cycles dans le système de fichiers). Seuls les liens symboliques fonctionnent sur les répertoires.
5. Dans `ls -la`, les liens symboliques sont affichés avec `->` indiquant la cible (ex. : lien `-> /chemin/cible`). Les liens physiques n'ont pas d'indicateur spécial, mais leur compteur de liens (2ème colonne) est `> 1`.

Remarque — Inœud et liens

Chaque fichier sur disque a un inœud (inode) qui contient les métadonnées. Un lien physique est une entrée de répertoire supplémentaire pointant vers le même inœud. Un lien symbolique est un fichier spécial contenant le chemin de la cible.

Correction 5.7 — Exercice pratique : Gestion de fichiers

1. Créer l'arborescence :

```
mkdir -p /home/etudiant/Documents/Rapports
mkdir /home/etudiant/Scripts
mkdir /home/etudiant/Sauvegarde
```

2. Copier le fichier :

```
cp /etc/hostname /home/etudiant/Documents/
```

3. Créer le lien symbolique :

```
ln -s /home/etudiant/Documents/hostname /home/etudiant/lien_hostname
```

4. Déplacer le fichier :

```
mv /home/etudiant/Documents/hostname /home/etudiant/Sauvegarde/
```

5. Donner les permissions 644 :

```
chmod 644 /home/etudiant/Sauvegarde/hostname
```

6. Vérifier le lien brisé :

```
ls -la /home/etudiant/lien_hostname
# Affiche un lien brisé (cible inexistante)
```

Le lien symbolique pointe toujours vers /home/etudiant/Documents/hostname mais ce fichier a été déplacé, donc le lien est brisé.

7. Supprimer le dossier Scripts :

```
rm -rf /home/etudiant/Scripts
```

6 Édition et traitement de données textuelles

Correction 6.1 — QCM : Éditeurs et filtres

1. Éditeur terminal le plus courant : **b)** vim — vim (Vi IMproved) est l'éditeur en mode terminal le plus installé par défaut. nano est plus simple mais moins puissant. gedit est graphique.
2. 10 premières lignes : **a)** head -10 fichier — head affiche le début du fichier. -n 10 ou -10 spécifie le nombre de lignes.
3. grep sert à : **b)** **Rechercher des motifs dans des fichiers** — grep (Global Regular Expression Print) filtre les lignes correspondant à un motif.

4. Édition batch : **b)** `sed` — `sed` (Stream Editor) transforme le texte en flux sans interaction. `awk` est aussi un outil de traitement mais plus orienté colonnes.

Correction 6.2 — Vrai ou Faux : Grep, sed, awk

1. **Vrai.** `grep -i` ignore la casse (insensible à la majuscule/minuscule).
2. **Vrai.** `sed -i` modifie le fichier directement (« in place »). Sans `-i`, les modifications s'affichent sur la sortie standard uniquement.
3. **Faux.** `awk` est un langage de programmation complet (conditions, boucles, fonctions, calculs), pas seulement un afficheur de colonnes.
4. **Vrai.** `grep -r` (ou `-R`) recherche récursivement dans tous les fichiers d'un répertoire.
5. **Faux.** `sed 's/old/new/'` ne remplace que la **première** occurrence sur chaque ligne. Pour toutes les occurrences, il faut `sed 's/old/new/g'` (flag `g` = global).
6. **Vrai.** `awk` utilise les expressions régulières étendues (ERE) par défaut.

Remarque — Flag `g` de `sed`

Sans le flag `g`, `sed 's/old/new/'` ne remplace que la première occurrence de chaque ligne. `sed 's/old/new/g'` remplace toutes les occurrences. `sed 's/old/new/2'` remplace uniquement la 2ème occurrence.

Correction 6.3 — Exercices avec `grep`

Soit le fichier `employees.csv` (séparateur virgule) :

1. Afficher les lignes contenant Informatique :

```
grep 'Informatique' employees.csv
```

Résultat : Dupont, Bernard et Durand (département Informatique).

2. Afficher les lignes contenant Paris (insensible à la casse) :

```
grep -i 'paris' employees.csv
```

3. Afficher les lignes ne contenant PAS Informatique :

```
grep -v 'Informatique' employees.csv
```

4. Compter les lignes contenant RH :

```
grep -c 'RH' employees.csv
```

Résultat : 2 (Martin et Lefevre).

5. Afficher les lignes commençant par D :

```
grep '^D' employees.csv
```

Résultat : Dupont et Durand.

6. Afficher les lignes contenant Paris ou Lyon :

```
grep -E 'Paris|Lyon' employees.csv
```

Ou : `grep 'Paris\\|Lyon' employees.csv` (sans `-E`, échapper le `|`).

Correction 6.4 — Exercices avec sed

1. Remplacer Paris par PARIS :

```
sed 's/Paris/PARIS/g' employees.csv
```

2. Supprimer les lignes contenant RH :

```
sed '/RH/d' employees.csv
```

3. Remplacer le premier champ en majuscules :

```
sed 's/^[^,]*\\U&/' employees.csv
```

Explication : `^[^,]*` capture le premier champ (jusqu'à la virgule), `\\U&` le convertit en majuscules.

4. Afficher uniquement les lignes 2 à 4 :

```
sed -n '2,4p' employees.csv
```

5. Remplacer les virgules par des points-virgules :

```
sed 's/,/;/g' employees.csv
```

6. Insérer une ligne après la ligne 1 :

```
sed '1 a\\Nouveau,Ventes,3000,Bordeaux' employees.csv
```

Correction 6.5 — Exercices avec awk

1. Afficher uniquement les noms (colonne 1) :

```
awk -F',' ' {print $1}' employees.csv
```

2. Noms et salaires des employés en Informatique :

```
awk -F',' ' $2 == "Informatique" {print $1, $3}' employees.csv
```

Résultat : Dupont 3500, Bernard 4200, Durand 3900.

3. Salaire moyen de tous les employés :

```
awk -F',' ' NR>1 {somme+= $3; nb++} END {print "Moyenne:", somme/nb}' employees.csv
```

Calcul : $(3500 + 2800 + 4200 + 3100 + 3900 + 2600)/6 = 3350$

4. Employés gagnant plus de 3000 :

```
awk -F',' ' NR>1 && $3 > 3000 {print $1, $3}' employees.csv
```

5. Nombre d'employés par département :

```
awk -F',' ' NR>1 {tab[$2]++} END {for (d in tab) print d, tab[d]}' employees.csv
```

Résultat : Informatique 3, RH 2, Ventes 1.

6. Employés de Paris avec salaire, triés par salaire décroissant :

```
awk -F',' 'NR>1 && $4 == "Paris" {print $1, $3}' employees.csv | sort -t' ' -k2 -nr
```

Résultat : Bernard 4200, Dupont 3500, Lefevre 2600.

Remarque — NR et END dans awk

NR (Number of Records) est le numéro de ligne courant. NR>1 permet de sauter l'en-tête. Le bloc END s'exécute après la lecture de toutes les lignes, idéal pour les calculs finaux (moyennes, totaux).

7 Quelques outils de traitement de données

Correction 7.1 — QCM : Outils de traitement

1. wc -l compte : **b) Les lignes** — wc (word count) avec -l compte les lignes, -w les mots, -c les octets.
2. Tri numérique croissant : **b) sort -n fichier** — Sans -n, le tri est alphabétique (« 10 » avant « 2 »!).
3. uniq : **b) Supprime les lignes consécutives en doublon** — Important : uniq ne fonctionne que sur des lignes consécutives. Il faut trier d'abord : sort | uniq.
4. cut -d',' -f2 extrait : **a) La 2ème colonne avec la virgule comme séparateur** — -d définit le délimiteur, -f le(s) champ(s).
5. tee permet de : **b) Lire l'entrée standard et écrire sur la sortie standard et dans un fichier** — Permet de voir le résultat à l'écran tout en le sauvegardant dans un fichier.

Correction 7.2 — Vrai ou Faux : Redirections et pipes

1. **Vrai.** > écrase le fichier existant (ou le crée).
2. **Vrai.** >> ajoute à la fin du fichier sans écraser.
3. **Vrai.** 2> redirige le flux d'erreur (stderr, descripteur 2) vers un fichier.
4. **Vrai.** Le pipe | connecte la sortie standard (stdout) de cmd1 à l'entrée standard (stdin) de cmd2.
5. **Faux.** cmd1 | cmd2 exécute les commandes **en parallèle** (pipeline), pas séquentiellement. Pour l'exécution séquentielle, on utilise cmd1 ; cmd2 ou cmd1 && cmd2.
6. **Vrai.** < redirige le contenu d'un fichier vers stdin (ex. : mail user < message.txt).
7. **Vrai.** &> (ou 2>&1) redirige à la fois stdout et stderr.

Remarque — Descripteurs de fichiers

Linux utilise 3 flux standards : stdin (0), stdout (1), stderr (2). Redirections : > = 1>, 2> = stderr, &> = stdout+stderr, 2>&1 = rediriger stderr vers stdout.

Correction 7.3 — Pipeline de commandes

1. Compter les fichiers .txt du répertoire courant :

```
ls *.txt | wc -l
```

Ou plus robuste : `find . -maxdepth 1 -name "*.txt" | wc -l`

2. Les 5 processus consommant le plus de mémoire :

```
ps aux --sort=-%mem | head -6
```

(6 lignes car l'en-tête est incluse ; ou `ps aux --sort=-%mem | tail -n +2 | head -5`)

3. Fichiers .log modifiés il y a moins de 3 jours, les compter :

```
find /var/log -name "*.log" -mtime -3 | wc -l
```

4. Extraire la 3ème colonne d'un CSV (séparateur ;), trier, dédupliquer :

```
cut -d ';' -f3 fichier.csv | sort | uniq
```

5. Utilisateurs connectés et nombre de sessions :

```
who | awk '{print $1}' | sort | uniq -c | sort -rn
```

6. Compter les occurrences de ERROR par fichier .log :

```
grep -r -c "ERROR" /var/log/*.log | grep -v ':0$'
```

Ou : `find /var/log -name "*.log" -exec grep -c ERROR {} \;`

Correction 7.4 — Exercice combiné

Soit le fichier notes.txt (séparateur ;).

1. Note moyenne en Maths :

```
grep ';Maths;' notes.txt | awk -F ';' ' {somme+= $3; n++}
END {print "Moyenne Maths:", somme/n}' notes.txt
```

Ou en une seule commande :

```
awk -F ';' ' $2=="Maths" {s+= $3; n++}
END {print "Moyenne:", s/n}' notes.txt
```

Calcul : $(15 + 9 + 14)/3 = 12,67$

2. Élèves ayant au moins une note < 10 :

```
awk -F ';' ' NR>1 && $3 < 10 {print $1}' notes.txt | sort -u
```

Résultat : Bernard (note 9 en Maths).

3. Nombre d'élèves différents :

```
awk -F ';' ' NR>1 {print $1}' notes.txt | sort -u | wc -l
```

Résultat : 3 (Dupont, Martin, Bernard).

4. Note maximale et pour qui :

```
awk -F';' 'NR>1 && $3 > max {max=$3; nom=$1; mat=$2}  
END {print nom, max, mat}' notes.txt
```

Résultat : Dupont, 18, Physique.

5. Notes de Dupont triées par matière :

```
grep 'Dupont' notes.txt | sort -t';' -k2
```

Résultat : Anglais 16, Maths 15, Physique 18.

8 Introduction à la programmation shell Bash

Correction 8.1 — QCM : Scripts Bash

1. Première ligne d'un script : **a) `#!/bin/bash`!** — C'est le « shebang » qui indique l'interpréteur à utiliser. `#!/bin/sh` est aussi valide mais moins spécifique.
2. Rendre un script exécutable : **d) Les réponses a et b** — `chmod +x` et `chmod 755` rendent le script exécutable. `exec` est une commande interne du shell, pas appropriée ici.
3. Déclarer une variable : **b) `var=valeur`** — En Bash, pas d'espaces autour du `=`. `declare` existe mais n'est pas la syntaxe de base. `set` sert à modifier les options du shell.
4. Code de retour : **a) `$?`** — `$?` contient le code de retour (0 = succès, non-0 = erreur). `$$` = PID du shell, `$#` = nombre d'arguments.

Correction 8.2 — Vrai ou Faux : Variables et structures

1. **Faux.** En Bash, les variables ne sont pas typées par défaut — ce sont des chaînes de caractères. On peut utiliser `declare -i` pour forcer un type entier, mais c'est optionnel.
2. **Vrai.** `$0` contient le nom (ou le chemin) du script en cours d'exécution.
3. **Vrai.** `$1` à `$9` contiennent les 9 premiers arguments positionnels. Au-delà, utiliser `${10}`, `${11}`, etc.
4. **Vrai.** `$#` contient le nombre d'arguments positionnels passés au script (sans compter `$0`).
5. **Vrai.** Les guillemets doubles (`"$var"`) développent les variables mais protègent les espaces et les caractères spéciaux de la séparation de mots.
6. **Faux.** Les apostrophes (`'$var'`) **ne développent pas** les variables — `$var` reste littéral. C'est la différence clé entre « " » et « ' ».
7. **Faux.** `$@` contient tous les arguments comme des **chaînes séparées** (un par argument). `$*` les contient comme une seule chaîne (sans guillemets). Avec guillemets : `"$@"` = arguments séparés, `"$*"` = une seule chaîne.

Remarque — Guillemets doubles vs apostrophes

`"$var"` → la variable est remplacée par sa valeur. `'$var'` → la chaîne littérale `$var` est affichée. Toujours préférer les guillemets doubles pour protéger les variables contenant des espaces.

Correction 8.3 — Variables et paramètres

Soit l'appel : `./script.sh Linux Admin 2026`

1. `$0` = `./script.sh` (nom/chemin du script)
2. `$1` = `Linux` (1er argument)
3. `$2` = `Admin` (2ème argument)
4. `$#` = 3 (nombre d'arguments, sans compter `$0`)
5. `$@` = `Linux Admin 2026` (tous les arguments, séparés)
6. `$?` = Code de retour de la dernière commande exécutée avant cette lecture (variable selon le contexte)
7. `$$` = PID du processus shell courant (numéro unique)

Correction 8.4 — Structures de contrôle

1. Vérifier si un fichier existe et afficher sa taille :

```
#!/bin/bash
if [ -f "$1" ]; then
    taille=$(stat -c%s "$1")
    echo "Le fichier $1 existe."
    echo "Taille : $taille octets"
else
    echo "Erreur : le fichier $1 n'existe pas."
    exit 1
fi
```

2. Pair ou impair :

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage : $0 <nombre>"
    exit 1
fi
if [ $(( $1 % 2 )) -eq 0 ]; then
    echo "$1 est pair"
else
    echo "$1 est impair"
fi
```

3. Compter les lignes de chaque fichier .txt :

```
#!/bin/bash
for fichier in *.txt; do
    if [ -f "$fichier" ]; then
        lignes=$(wc -l < "$fichier")
        echo "$fichier : $lignes lignes"
    fi
done
```

4. Nombres de 1 à 10 (boucles for et while) :

```
#!/bin/bash
echo "Avec for :"
```



```
for i in {1..10}; do
    echo -n "$i "
done
echo
echo "Avec while :"
```

```
n=1
while [ $n -le 10 ]; do
    echo -n "$n "
    n=$((n + 1))
done
echo
```

5. Fonction factorielle :

```
#!/bin/bash
factorielle() {
    local n=$1
    local result=1
    if [ $n -le 1 ]; then
        echo 1
        return
    fi
    for ((i=2; i<=n; i++)); do
        result=$((result * i))
    done
    echo $result
}
if [ $# -ne 1 ]; then
    echo "Usage : $0 <nombre>"
    exit 1
fi
echo "$1! = $(factorielle $1)"
```

Correction 8.5 — Conditions et tests

1. Fichier existe : [-e fichier.txt] ou test -e fichier.txt
2. Fichier exécutable : [-x script.sh] ou test -x script.sh
3. Variable non vide : [-n "\$nom"] ou test -n "\$nom"
4. Nombre > 10 : ["\$n" -gt 10] ou test "\$n" -gt 10
5. Répertoire : [-d rep/] ou test -d rep/
6. Chaîne égale : ["\$reponse" = "oui"] ou test "\$reponse" = "oui"
7. Fichier plus récent : [fichier.txt -nt sauvegarde.txt]
8. Nombres différents : ["\$a" -ne "\$b"]

Remarque — Syntaxe [[]] vs []

[[]] est une version améliorée de [] (spécifique à Bash). Elle supporte && et || au lieu de -a et -o, le pattern matching ([[\$s == *.txt]]), et évite certains problèmes de substitution de mots. Préférer [[]] en Bash.

Correction 8.6 — Projet : Script de sauvegarde

```
#!/bin/bash
# sauvegarde.sh - Script de sauvegarde

# Fonction : vérifier l'existence du répertoire source
verifier_source() {
    if [ ! -d "$1" ]; then
        echo "Erreur : le repertoire '$1' n'existe pas."
        exit 1
    fi
}

# Fonction : créer le répertoire de sauvegarde
creer_backup() {
    local date=$(date +%Y%m%d)
    local dest="/tmp/backup_${date}"
    mkdir -p "$dest"
    echo "$dest"
}

# Fonction : copier et compter
copier_sauvegarde() {
    local source="$1"
    local dest="$2"
    cp -r "$source"/* "$dest"/ 2>/dev/null
    local nb_fichiers=$(find "$dest" -type f | wc -l)
    local taille=$(du -sh "$dest" | cut -f1)
    echo "Sauvegarde terminée !"
    echo "Fichiers copiés : $nb_fichiers"
    echo "Taille totale : $taille"
}

# Programme principal
if [ $# -ne 1 ]; then
    echo "Usage : $0 <repertoire_source>"
    exit 1
fi

verifier_source "$1"
dest=$(creer_backup)
echo "Destination : $dest"
copier_sauvegarde "$1" "$dest"
```

Remarque — Améliorations possibles

Pour un script de production, on pourrait ajouter : (1) vérification de l'espace disque avec `df`, (2) compression avec `tar czf`, (3) rotation des sauvegardes (suppression des sauvegardes trop anciennes), (4) journalisation dans un fichier log, (5) vérification d'intégrité avec `md5sum`.